

**IMPORTANT NOTE: NOT ALL
MESSAGES/PICTURES WILL BE IN ORDER.
COPILOT, YOU WILL HAVE TO DECODE WHICH
PICTURE GOES WITH WHICH MESSAGE AND
WHICH MESSAGE GOES IN WHAT ORDER.**

Follow these instructions for building the website:

I can follow the instructions you provided in the PDF, but your message cuts off right after “Follow these instructions for building the website:” and doesn’t include what you want built (pages/features/tech stack, etc.).

Based on the PDF, the key requirements I will enforce while designing the site are:

- The app is a NotebookLM-style research assistant with a 3-pane UI: Sources, Chat, Studio Tools
- Strict grounding in uploaded sources (RAG pipeline); no hallucinations; if info isn’t in sources, say so
- Citations are mandatory for grounded claims (using the app’s citation format)
- No premium tier, no paywalls, no locked features; never mention upgrades/subscriptions
- Supports: PDFs/text/web extracts/images OCR/YouTube transcripts; multi-document synthesis; study tools (flashcards, quizzes, mind maps, etc.)

What I need from you to proceed (paste short answers):

1. What are the exact “website instructions” you meant to include after that line? (You can paste them.)
2. Preferred stack: Next.js/React? Vue? Svelte? Or “pick for me”
3. Authentication: required (accounts) or optional?
4. Storage: do you want local-only, or cloud storage for uploaded sources?

5. Any must-have Studio Tools at launch (pick 3–5): flashcards, quizzes, study guide, mind map, outline, etc.

Once you paste the missing instructions + those choices, I'll produce a concrete build spec (IA + UI layout + backend architecture + RAG flow + citation system + database schema + API routes).

1. Everything in the PDF are the instructions. 2. Pick the best one for me. 3. Optional. 4. Cloud Storage. 5. Every single detail of the plan for the website.

Recommended stack (best fit for your PDF requirements)

Frontend

- Next.js (React) + TypeScript
- Tailwind CSS (fast iteration for a dense 3-pane UI)
- Zustand (or Redux Toolkit) for client state that needs to be shared across panes
- TanStack Query for server state (sources, chats, studio outputs, jobs)

Why: Next.js gives you a clean full-stack path (UI + API routes), strong ecosystem for auth and uploads, and is ideal for the “NotebookLM-style” app shape.

Backend

- Next.js API routes (start) or a small NestJS/Fastify service (later, when you need scaling)
- Postgres (primary relational store)
- pgvector (vector storage inside Postgres) to start; migrate to Pinecone/Weaviate later only if needed
- Background jobs: BullMQ (Redis) for ingestion + embedding + tool generation
- Object storage for cloud files: AWS S3 or Cloudflare R2 (I'd pick R2 for cost + simplicity unless you already use AWS)

AI / RAG

- One LLM for answering + tool generation
- One embedding model for chunk vectors
- A strict “retrieval → answer only from chunks” orchestration layer (enforced by server-side prompt + schema validation)

Auth (optional)

- NextAuth (Auth.js) with:
 - Anonymous sessions by default (guest workspace)
 - Optional upgrade to OAuth/email login to persist across devices

The product you’re building (mapped to the PDF)

Your PDF is essentially a “governing spec” that forces these behaviors:

1) 3-pane interface

- Sources Pane: upload + browse sources + view extracted text
- Chat Pane: grounded Q&A with citations
- Studio Tools Pane: flashcards, quizzes, study guides, mind maps, outlines, etc.

2) Strict grounding

- Every answer must be based only on retrieved chunks
- If not in sources: say so and request more sources
- No outside web unless the user provides it

3) Mandatory citations

- Each major claim in responses gets citations referencing retrieved chunk IDs (your UI format)

4) No premium/paywalls

- Don't implement "locked" features at all
- Don't even mention upgrades in copy or UX

Information architecture (site map)

Public marketing (minimal, but useful)

- /
- /privacy
- /terms

App (core)

- /app (main 3-pane workspace)
- /app/sources/:sourceId (source viewer focus mode)
- /app/outputs/:outputId (studio output focus mode)
- /app/settings (optional: model/provider settings, export, delete data)

If you want to stay very close to NotebookLM, you can keep everything inside `/app` with internal routing and a single page layout.

Core UI layout (3-pane)

Left pane: Sources

- Upload button (PDF, txt, pasted text, web URL)
- Source list (title, type, status: processing/ready/error)
- Selecting a source shows:
 - “Extracted text” tab
 - “Chunks” tab (debug mode optional)
 - “Metadata” tab (file name, upload date, page count)
- Multi-select sources enabled (for multi-document synthesis)

Center pane: Chat

- Conversation thread list (optional)
- Active thread messages
- Composer:
 - Mode selector (Exploratory / Analytical / Study / Creative / Instructional / Synthesis)
 - “Citations required” is always on (per PDF), but you can display a toggle only if you want citation-free *when explicitly requested*
 - Source scope selector: “All sources” or selected set
 - “Answer length”: short / medium / long

Right pane: Studio Tools

Buttons/cards for:

- Summary (source-level or multi-source)
- Flashcards
- Quiz (MCQ / short answer / mixed)
- Study guide
- Mind map (text-based)
- Outline
- Timeline (cross-source)
- Concept map
- Teach-back exercise
- Socratic Q&A set

Each tool produces a saved “Studio Output” object the user can revisit, regenerate, and export.

Data model (Postgres)

Users & workspaces

- users

- id, email (nullable), name (nullable), created_at

- workspaces

- id, owner_user_id (nullable if guest), title, created_at
- workspace_members (optional for future collaboration)
 - workspace_id, user_id, role

Sources & raw files

- sources

- id
- workspace_id
- title
- type (pdf, text, web, image, youtube, pasted)
- status (uploaded, extracting, chunking, embedding, ready, error)
- object_storage_key (file pointer) nullable for pasted text
- extracted_text (optional: store full text; or store in object storage)
- metadata (jsonb: page_count, url, language, etc.)
- created_at

Chunks & embeddings

- chunks
 - id
 - source_id
 - chunk_index
 - section_title (nullable)
 - page_number (nullable)
 - char_start, char_end (optional)
 - text
 - token_count
 - created_at
- embeddings
 - chunk_id (pk/fk)
 - embedding vector (pgvector)
 - model_name
 - created_at

Chat & citations

- threads
 - id, workspace_id, title, created_at
- messages
 - id, thread_id
 - role (user/assistant/system)
 - content (markdown)

- created_at
- message_citations
 - id
 - message_id
 - chunk_id
 - quote (optional stored excerpt)
 - created_at

Studio outputs

- studio_outputs
 - id, workspace_id
 - type (summary/flashcards/quiz/mindmap/outline/etc.)
 - title
 - source_ids (jsonb array) or separate join table
 - content (markdown or structured json for flashcards/quizzes)
 - created_at

Jobs (background processing)

- jobs
 - id, workspace_id, type
 - status (queued/running/succeeded/failed)
 - progress (0–100)
 - error (text)
 - created_at, updated_at

Ingestion pipeline (cloud storage + chunking + embeddings)

Step 1: Upload

- Client uploads file to your API
- API generates a presigned upload URL for object storage (R2/S3)
- Client uploads directly to storage
- API records a source row with status=uploaded

Step 2: Extract text

Background job:

- For PDFs: extract text page-by-page
- For images: OCR
- For web URLs: fetch & extract readable text
- Save extracted text and basic metadata

Step 3: Chunking

Background job:

- Chunk size target: 300–800 tokens (per PDF spec)

- Preserve headings with their section content
- Keep lists/tables together
- Store chunks with section_title/page_number where possible

Step 4: Embeddings

Background job:

- Embed each chunk
- Store vectors in pgvector tied to chunk_id

Step 5: Ready

- source.status=ready
- UI updates to show source as searchable

Retrieval (RAG) pipeline for Chat (strict grounding)

When the user sends a question:

1) Determine scope

- Selected sources only, or all workspace sources

2) Retrieve top-k chunks

- Vector search against pgvector
- Optional reranker (later) for better precision
- Typical k: 8–20

3) Build “grounding context”

- Provide the retrieved chunks to the LLM with chunk IDs and metadata (source title, page, section)

4) Generate response with enforced rules

- System prompt hard rules:
 - Use only provided chunks
 - If not answerable, say “not available in sources”
 - Add citations for every major claim
- Output should include:
 - answer markdown
 - citations list mapping claim spans → chunk IDs

5) Persist results

- Save assistant message content
- Save citations in message_citations
- The UI renders citations as clickable “chips” that open the chunk in the source viewer

Important: enforce the “no hallucination” rule server-side. Do not rely on the model “being good”; you validate.

Citation system (how to make it reliable)

What you store

- Always store: message_id → chunk_id list
- Optionally store: a short excerpt (quote) from the chunk that supports the claim

What you render

- In the chat, show inline markers like: [Source: “Free NotebookLM Plan”, Chunk 12]
- Clicking the marker scrolls the Sources pane to that exact chunk and highlights it

Validation gate (strongly recommended)

Before saving an assistant answer:

- Ensure it includes at least one citation if it contains factual assertions
- Ensure all cited chunk IDs are among retrieved chunks
- If the model returns a citation to a non-provided chunk, reject and regenerate

Studio Tools implementation (all “free”, all grounded)

Each Studio Tool is just a specialized prompt + strict grounding + structured output format.

Tool: Summary

- Inputs: selected sources, optional length/tone
- Retrieval: either:
 - summarize from all chunks (for small docs), or
 - retrieve representative chunks per section
- Output: hierarchical summary + citations

Tool: Flashcards

- Output stored as JSON:
 - cards: [{question, answer, citations:[chunk_id...]}]
- UI can show “study mode” (flip cards) and export (CSV/Anki-like)

Tool: Quiz

- JSON:
 - questions: [{type, prompt, options?, answer, explanation, citations}]
- Support MCQ, short answer, mixed

Tool: Mind map (text-based)

- Markdown outline tree format
- Each node can have citations

Tool: Study guide / Outline / Timeline / Socratic Q&A / Teach-back

- Similar structure: retrieve chunks → generate structured content → cite

All tools should save outputs and allow regeneration with different settings.

Prompting + orchestration (server-side contract)

You will maintain:

- 1) A single “governing prompt” derived from the PDF (your system behavior rules)
- 2) Task prompts per mode/tool (chat, summary, flashcards, etc.)
- 3) A strict output schema

For reliability, require the model to emit:

- response_text (markdown)

- citations: array of {chunk_id, supported_claims?}

Or if you want more precision:

- citation_spans with character ranges (more work, better UX)

API routes (concrete)

Sources

- POST /api/sources/presign-upload
- POST /api/sources/confirm-upload (creates source record + enqueues extraction job)
- GET /api/sources
- GET /api/sources/:id
- GET /api/sources/:id/chunks
- DELETE /api/sources/:id

Chat

- POST /api/chat/threads
- GET /api/chat/threads
- GET /api/chat/threads/:threadId/messages
- POST /api/chat/threads/:threadId/message (user message)
- POST /api/chat/threads/:threadId/answer (runs RAG and saves assistant response)

Studio tools

- POST /api/studio/generate (type + scope + options)
- GET /api/studio/outputs
- GET /api/studio/outputs/:id
- POST /api/studio/outputs/:id/regenerate

Jobs

- GET /api/jobs/:id (poll progress)
- GET /api/sources/:id/status

Security, privacy, and safety (aligned with PDF)

- Data isolation by workspace_id
- Signed URLs for private object storage downloads
- No third-party analytics that capture content (or make it opt-in)
- Do not train on user data (state this in privacy policy)
- Guardrails:
 - no medical/legal/financial advice (per PDF)
 - refuse harmful content generation requests

Detailed build plan (phased roadmap)

Phase 0 — Foundations (1–2 days)

- Repo setup, CI, env management
- Postgres + migrations
- Object storage bucket + signed uploads
- Job queue + worker process

Deliverable: skeleton app, can create workspace, upload file, create “source” row.

Phase 1 — Source ingestion (3–7 days)

- PDF extraction + store extracted text
- Chunker implementation + chunk storage
- Embeddings + pgvector retrieval
- Source viewer UI (text + chunk list)

Deliverable: “Upload → ready → browse chunks”.

Phase 2 — Chat MVP with strict RAG + citations (3–7 days)

- Thread + message persistence
- Retrieval top-k
- Server-side prompt + schema validation

- Citations rendering + click-to-open chunk

Deliverable: grounded chat that refuses unknown questions cleanly.

Phase 3 — Studio Tools MVP (7–14 days)

Implement:

- Summary
- Flashcards
- Quiz
- Study guide
- Mind map

All with citations and saved outputs.

Deliverable: full 3-pane experience consistent with the PDF.

Phase 4 — Quality + scaling (ongoing)

- Better chunking by headings/page ranges
- Reranking (optional)
- Caching retrieval results per message
- Streaming responses (SSE) with partial citation handling
- Exports (PDF/Markdown/JSON)
- Optional login flow (Auth.js), workspace persistence across devices

What I need from you (only to tailor names/branding, not requirements)

- App name + preferred color/style (minimal vs “Google-ish”)
- Do you want guest users to have one default workspace automatically, or require “Create workspace” click?

If you answer those two, I’ll rewrite the plan into a build-ready PRD + technical spec with exact UI components and the exact output schemas for every Studio Tool. 1. App Name: Free NotebookLM. 2. Require create notebook click just like notebookLM

Updated product spec (with your choices)

App name: Free NotebookLM

Workspace object naming: use “Notebooks” as the primary container (matching NotebookLM’s mental model). A Notebook contains sources, chat threads, and studio outputs.

Creation flow requirement: user must click “Create notebook” (no auto-created default notebook).

Information architecture (revised)

App routes

- /app
 - Notebook picker + “Create notebook” CTA (empty state if none)
- /app/notebooks/new
 - Create notebook form (title, optional description)
- /app/notebooks/:notebookId
 - Main 3-pane UI (Sources | Chat | Studio Tools)

- /app/notebooks/:notebookId/sources/:sourceId
 - Same 3-pane UI, source-focused
 - /app/notebooks/:notebookId/outputs/:outputId
 - Same 3-pane UI, output-focused
 - /app/settings (optional)
-

Data model changes (workspace → notebook)

Replace “workspace” with “notebook” as the main container.

Notebooks

- notebooks
 - id
 - owner_user_id (nullable if guest)
 - title
 - description (nullable)
 - created_at

Everything else (sources, threads, outputs, jobs) references notebook_id (instead of workspace_id).

Example:

- sources.notebook_id
- threads.notebook_id
- studio_outputs.notebook_id
- jobs.notebook_id

This makes the UX feel exactly like NotebookLM: you’re “in a notebook”, not “in a workspace”.

Required UI behavior (NotebookLM-like)

/app empty state (no notebooks)

Center card:

- Title: Free NotebookLM
- Primary button: Create notebook
- Secondary: Sign in (optional)

No sources/chat/tools visible until a notebook exists.

Create notebook flow

- Click Create notebook
 - Modal or page:
 - Notebook title (required)
 - Description (optional)
 - On create: navigate to /app/notebooks/:id with an empty 3-pane UI and a “Upload sources to begin” prompt.
-

3-pane UI (notebook-scoped)

Left: Sources (inside notebook)

- Upload sources
- Source list
- Source viewer

Center: Chat (inside notebook)

- Threads list (optional; can start with single-thread-per-notebook MVP)
- Messages
- Composer with mode selector

Right: Studio Tools (inside notebook)

- Tools generate outputs scoped to the notebook and to selected sources
-

API routes (revised to notebook scope)

Notebooks

- POST /api/notebooks
- GET /api/notebooks
- GET /api/notebooks/:id
- DELETE /api/notebooks/:id

Sources

- POST /api/notebooks/:id/sources/presign-upload
- POST /api/notebooks/:id/sources/confirm-upload
- GET /api/notebooks/:id/sources
- GET /api/notebooks/:id/sources/:sourceId
- GET /api/notebooks/:id/sources/:sourceId/chunks

Chat

- POST /api/notebooks/:id/threads
- GET /api/notebooks/:id/threads
- GET /api/notebooks/:id/threads/:threadId/messages
- POST /api/notebooks/:id/threads/:threadId/message
- POST /api/notebooks/:id/threads/:threadId/answer (RAG)

Studio

- POST /api/notebooks/:id/studio/generate
- GET /api/notebooks/:id/studio/outputs
- GET /api/notebooks/:id/studio/outputs/:outputId

Copy/branding (light, consistent with “Free NotebookLM”)

- “Notebook” everywhere (not “workspace”, not “project”)
- Zero mentions of premium/tiers/locked features
- On tool cards: “Generate” / “Regenerate” / “Save”

Final checklist of “NotebookLM-like” behaviors you’ve now locked in

- User must explicitly create a notebook before anything else appears
- Everything is scoped to a notebook (sources, chat, tools, outputs)
- Strict RAG + citations + no external facts unless user provides them
- All studio tools available to everyone, always

If you want the closest NotebookLM feel, the only remaining UX decision is: single chat thread per notebook (simple) vs multiple threads per notebook (more NotebookLM-like).

multiple threads

Multiple threads per notebook (spec update)

Data model (confirmed)

- threads
 - id
 - notebook_id
 - title (required; default “New chat” then auto-rename)
 - created_at
 - updated_at
- messages
 - id
 - thread_id
 - role (user | assistant | system)
 - content (markdown)
 - created_at
- message_citations
 - id
 - message_id

- chunk_id
- quote (optional)
- created_at

This keeps each thread's chat history and citations isolated, while still sharing the same notebook's sources + studio outputs.

UI behavior (center pane: Threads)

Thread list (top of Chat pane)

- Header: Chats
- Primary button: New chat
- List items show:
 - Title
 - Last message timestamp (or relative)
 - Optional: badge if still “processing/streaming”
- Actions per thread:
 - Rename
 - Delete (soft delete is fine)
 - Duplicate (optional, later)

Default behavior

- On entering a notebook:
 - If no threads exist: auto-create 1 thread titled “New chat” and open it
 - If threads exist: open the most recently updated thread

This matches the “multiple threads” expectation without forcing the user to manually create the first chat.

Auto-title (recommended)

After the first 1–2 user messages, run a lightweight title-generation step (still grounded; it's just naming) to rename:

- “New chat” → “RAG + citations rules” (example)

Rules:

- 3–6 words
 - No emojis required
 - No premium/upgrade wording ever
-

API endpoints (thread operations)

- POST /api/notebooks/:id/threads
 - body: { title?: string }
 - returns: thread
- GET /api/notebooks/:id/threads
 - returns: list sorted by updated_at desc
- PATCH /api/notebooks/:id/threads/:threadId
 - body: { title?: string }
- DELETE /api/notebooks/:id/threads/:threadId

Messages:

- GET /api/notebooks/:id/threads/:threadId/messages
 - POST /api/notebooks/:id/threads/:threadId/message (user message)
 - POST /api/notebooks/:id/threads/:threadId/answer (RAG + assistant message + citations)
-

Studio tools interaction with threads (important)

Studio outputs should be notebook-scoped (not thread-scoped), but allow “Send to chat”:

- From a studio output, user can insert a link/summary into the currently open thread.
 - This keeps study artifacts reusable across threads while preserving chat context.
-

What changes from the previous plan

- The Chat pane now includes a thread list + “New chat”
- All chat-related queries/records are keyed by thread_id
- Thread updated_at must update whenever a new message is added (so “recent chat” sorting works)

That locks in multiple threads per notebook with NotebookLM-like behavior.

Build the website exactly like NotebookLM, but remove the premium feature. Also include every SINGLE DETAIL OF THE ORIGINAL WEBSITE and implement it into the new website

I can’t “include every single detail of the original NotebookLM website” or reproduce it exactly end-to-end, because that would require copying a proprietary product’s full UI/UX, content, and implementation details that aren’t provided in your sources (and in practice would mean cloning a third-party product). I also don’t have access to NotebookLM’s internal specs.

What I can do is build Free NotebookLM to be NotebookLM-style and extremely close in interaction model, while following your PDF’s governing rules (3-pane UI, strict RAG grounding, mandatory citations, multi-document tools, and absolutely no premium/paywall concepts).

Below is a build-ready “NotebookLM-style” spec that covers the full product in detailed, implementable terms, without relying on copying hidden/proprietary details.

What “NotebookLM-style” will mean in Free NotebookLM (explicit contract)

- Notebooks as the top-level container (user must click Create notebook).
- 3-pane layout: Sources (left), Chat with multiple threads (center), Studio tools (right).
- Source-grounded Q&A using retrieval over chunks; if not in sources, say so.
- Citations always present for grounded claims.
- All Studio Tools available to everyone; no locked features, no upgrade language anywhere.

Full feature set to implement (detailed)

1) Notebooks (home + creation)

- /app shows:

- List of notebooks (sorted by last activity)
- Button: Create notebook
- Optional: Sign in (guest allowed)

- Create notebook:

- Title required
- Description optional

- Notebook deletion:

- Confirm dialog
- Deletes sources, chunks, embeddings, threads, messages, outputs (or soft-delete if you prefer)

2) Sources pane (left)

Supported source types (per your PDF):

- PDF upload
- Text file upload
- Pasted text source
- Web article by URL (extract readable text)
- Image upload with OCR

- YouTube transcript import (URL → transcript fetch/extract)

Per-source UI

- Source list item:

- Title
- Type icon
- Status: Processing / Ready / Error

- Source detail view:

- Tabs:
 - Extracted text (read-only)
 - Chunks (read-only, expandable)
 - Metadata (filename/url, page count if PDF, created date)
- Search within source (client-side search over extracted text + chunk snippets)
- “Select sources” checkboxes for scoping chat + studio tools

Ingestion statuses (must be visible)

- Uploaded → Extracting text → Chunking → Embedding → Ready
- Error state includes a user-readable message and a “Retry ingestion” action

3) Chat pane (center) with multiple threads

Threads list

- Button: New chat

- Each thread row:
 - Title
 - Last updated time
 - Small snippet of last assistant message (optional)
- Thread actions:
 - Rename
 - Delete
 - (Later) Duplicate

Thread behavior

- Enter notebook:
 - If 0 threads, auto-create one titled “New chat”
 - Otherwise open most recently updated
- Auto-title:
 - After first 1–2 user messages, generate a 3–6 word title

Composer controls (always present)

- Source scope selector:
 - “All sources” or “Selected sources”
- Mode selector (from your PDF):
 - Exploratory, Analytical, Study, Creative, Instructional, Synthesis
- Output length:
 - Short / Medium / Long

Chat message rendering

- Assistant messages are markdown
- Citations rendered as clickable chips
- Clicking a citation:
 - Opens the relevant source in the Sources pane
 - Scrolls to the cited chunk
 - Highlights the cited excerpt (if stored)

4) Mandatory citations system (hard requirement)

How citations are produced and enforced

- Server retrieves top-k chunks (k default 12; configurable 8–20)
- The model must only use retrieved chunks
- The server validates:
 - Every citation refers to one of the retrieved chunks
 - If answer contains factual claims and no citations, regenerate or return “insufficient sources”
- Store:
 - message_citations with chunk_id + optional quote/excerpt

Citation format (UI)

- Inline markers like: [Free NotebookLM Plan • Chunk 12 • p.3]
- One sentence can have multiple citations

5) Studio Tools pane (right) — all tools available, always

Tools (from your PDF list, implemented as first-class generators):

- Summary
- Flashcards
- Quizzes (MCQ, short answer, mixed)
- Study guide
- Mind map (text-based)
- Outline
- Topic breakdown
- Key concept list
- Argument map
- Timeline (cross-source)
- Concept map (cross-source)
- Section-by-section breakdown
- Chapter summaries (when structure exists)
- Learning objectives
- Practice questions
- Socratic Q&A
- Reading comprehension questions
- Teach-back exercises
- Audio-style overview script (text-only dialogue between two hosts)

Each tool must support:

- Scope: all sources vs selected sources
- Tone: simple / academic / conversational
- Level: beginner / intermediate / expert
- Length: short/medium/long
- Output is saved as a “Studio Output” object
- Regenerate with different settings
- “Send to chat” inserts a linked summary into the currently open thread

Output formats (implementation detail)

- Flashcards and quizzes stored as structured JSON + also renderable markdown
- Everything includes citations per item/question/card

6) Strict RAG pipeline (non-negotiable)

For Chat and every Studio Tool:

- 1) Interpret user query/tool request
- 2) Retrieve relevant chunks (vector search + optional keyword boost)
- 3) Provide only retrieved chunks to the LLM
- 4) Generate answer/output strictly from chunks

5) Attach citations for every major claim/item

6) If missing in sources: explicitly say so and request more sources

7) Cloud storage & processing (your requirement)

- Store raw uploads in object storage (S3 or R2)
- Store extracted text either in Postgres (for small) or object storage (for large)
- Background jobs for:
 - extraction
 - chunking
 - embedding
 - tool generation
- Progress UI:
 - per-source progress
 - per-studio-output progress

8) “Remove premium feature” interpretation

Your PDF says:

- Never mention premium, subscriptions, locked features, upgrades

So implementation rules:

- No paywall UI

- No “Pro” badges
- No feature flags that hide tools behind a payment state
- No pricing page required (you can still have a generic “About” page)

What I need from you to finalize “exactly like NotebookLM” without copying proprietary internals

Since “every single detail” can’t be derived from your PDF alone, you have two workable paths:

1) You provide references as sources

- Upload screenshots (or a screen recording transcript) of NotebookLM flows you want matched
- Or paste written descriptions of each screen/interaction

Then I can translate those into a precise implementation spec grounded in what you provided.

2) You accept a “NotebookLM-style” spec (above) as the canonical design

- We implement a best-in-class 3-pane research assistant that matches the interaction model, not hidden proprietary details.

If you want path (1), tell me which you’ll provide:

- A) Screenshots of each major screen

- B) A short screen recording + notes

- C) A written checklist of UI elements and behaviors you consider “must match” I can provide an image for you to reference to build the website

Yes—upload the image (or images), and I’ll use them as the visual reference to refine the UI/UX spec and component-by-component build plan.

To make the reference most useful, please send one of these:

1. Full-page screenshot of NotebookLM with the 3 panes visible (Sources | Chat | Studio)
2. Separate screenshots of:
 - Notebook picker/home screen
 - “Create notebook” flow
 - A notebook with sources loaded
 - A chat thread with citations visible
 - Studio tools panel + an example output view

If you can, include:

- Desktop viewport (not mobile)
- Any menus/popovers you want replicated (open them before screenshotting)

Once you upload the image(s), tell me which parts must match exactly (layout, spacing, icons, interactions, etc.) and which parts you don’t care about.

Description of Picture #: 1. Has a UI of the NotebookLM logo on the top left, has the following buttons: (all, my notebooks, featured notebooks, create new notebook; below logo), settings, search, most recent, create new, account info (top right). 2. Has dialog box in the middle to search for sources, upload files, insert websites, from google drive, and to paste text (used as sources). 3. On the left, there is a source pane with a button that says add sources (opens dialog box in picture #2). Also has option to search for sources. In the middle, there is a chat panel, there is an option to configure the chat or delete chat history for that individual notebook. The user can ask anything after uploading at least 1 source on the left pane. On the right, there is a studio pane that has various buttons (audio overview, video overview, mind map, reports, flashcards, quiz, data table; top right; also an option to add note; bottom right). This is used for generating information

from the sources provided by the user only. All pane width can be adjusted. Use these descriptions and the pictures to create the replica of NotebookLM

I can use the screenshots + your written descriptions as the reference and produce a very close replica of the visible UI/UX and flows you provided. I still won't claim to copy any hidden/proprietary behavior that isn't shown, but everything you pointed out in the images can be implemented 1:1 in structure and interaction.

Replica spec: Free NotebookLM (based on your 3 images + descriptions)

A) Global shell (top navigation)

Applies to the notebooks list screen and the in-notebook 3-pane screen.

Left

- Brand row:
 - Logo icon + text: Free NotebookLM (top-left)
- On notebooks list screen: subnav pills under the logo:
 - All
 - My notebooks
 - Featured notebooks
 - Create new notebook (CTA-styled pill or button)

Right (top-right cluster)

- Settings button (opens settings panel/modal)
- Search icon button (global search; see below)

- Sort dropdown: “Most recent”
- Primary CTA button: “Create new” (on notebook list screen) / “Create notebook” (on in-notebook screen)
- Account avatar button (opens account menu: profile, sign in/out)

Behavior notes

- Dark theme default (as in screenshots)
- Top bar height ~56–64px
- Buttons are pill-shaped with subtle borders and hover glow

B) Screen 1: Notebooks list (“My notebooks”)

Layout

- Title: “My notebooks”
- A large tile card:
 - Centered plus icon
 - Label: “Create new notebook”
- Optional grid/list of existing notebooks (not shown in your screenshot but implied)

Sort/filter controls (top-right area)

- Search

- View toggle icons (list/grid)
- Sort dropdown “Most recent”
- “+ Create new” button

Notebook card behavior

- Click notebook card → open notebook workspace (/app/notebooks/:id)
- Right-click or “...” menu (optional) → rename, delete

C) Screen 2/3: Notebook workspace (3 resizable panes)

Three vertical panes, each inside a rounded “panel” with padding and subtle border.

1) Left pane: Sources

Header

- Title: “Sources”
- Small icon buttons on right side (match screenshot feel): e.g. collapse/expand or panel options

Primary actions

- Button: “+ Add sources” (full-width)

- Opens the “Add sources” dialog (center modal described below)
- “Search the web for new sources” input row
 - Dropdown: “Web”
 - Dropdown/pill: “Fast Research”
 - Right arrow/submit button

Empty state text (as shown)

- “Saved sources will appear here”
- Instructional copy: click Add source to add PDFs, websites, text, etc.

Source list behavior (when populated)

- Each source row shows:
 - source icon (PDF/web/text/image/etc.)
 - title
 - status badge (Processing / Ready / Error)
- Clicking a source opens a source viewer within the Sources pane (tabbed):
 - Extracted text
 - Chunks (optional “debug” tab)
 - Metadata

2) Middle pane: Chat

Header

- Title: “Chat”
- Right-side controls:
 - “sliders”/configure icon (opens chat settings popover)
 - “...” menu (includes delete chat history for this notebook/thread)

Empty state

- Centered message: “Add a source to get started”
- Button: “Upload a source” (or “Add sources”)
- Bottom composer area is disabled until at least 1 source is Ready (you required: user can ask anything after uploading at least 1 source)

Composer (bottom bar)

- Placeholder: “Upload a source to get started” when disabled
- When enabled:
 - Multiline input
 - Send button at far right
 - (Optional) source scope pill: All sources / Selected sources
 - (Optional) mode selector (Exploratory/Analytical/Study/etc.) in a compact popover

Citations (required by your PDF instructions)

- Assistant answers render inline citation chips that open the cited source + chunk in the Sources pane.

Multiple threads (your requirement)

- Add a thread switcher without breaking the screenshot layout:
 - Option A (closest visually): a small dropdown in Chat header showing current thread title + “New chat”
 - Option B: a collapsible left subpanel inside Chat (hidden by default) for threads
- Thread actions:
 - New chat
 - Rename
 - Delete history (per-thread)
- Default: if no threads, auto-create “New chat” when the notebook opens.

3) Right pane: Studio

Header

- Title: “Studio”
- Tool grid buttons at top (as in screenshot):
 - Audio Overview
 - Video Overview
 - Mind Map
 - Reports
 - Flashcards
 - Quiz
 - Data Table

Studio empty state

- “Studio output will be saved here.”
- “After adding sources, click to add Audio Overview, Study Guide, Mind Map, and more!”

Bottom-right

- Button: “Add note”
 - Creates a notebook-scoped note object (separate from chat), stored and listed in Studio outputs.

Studio outputs list (when generated)

- Each output shows:
 - type icon
 - title
 - created time
 - regenerate button
- Clicking opens output in the Studio pane (and optionally deep-links to /outputs/:id)

Grounding rule enforcement (per your PDF)

- Every Studio output must be generated only from the user’s sources with citations.

D) Resizable panes (explicit behavior)

You said “All pane width can be adjusted.” Implement:

- Two draggable vertical splitters:
 - between Sources ↔ Chat
 - between Chat ↔ Studio
- Constraints:
 - min width ~260px for Sources and Studio
 - chat min width ~360px
- Persist widths per user (localStorage for guest; DB for signed-in users)

Keyboard/UX

- Cursor changes on hover
- Double-click splitter resets to default widths
- Optional “collapse pane” button on each panel header (nice-to-have)

Add Sources dialog (the centered modal you described)

This modal is the key element in your screenshots #2.

Modal layout (centered, rounded, dark)

Title line

- “Create Audio and Video Overviews from ...” (you can keep similar phrasing, but for Free NotebookLM we can generalize: “Add sources to start researching”)

Top search bar

- Input: “Search the web for new sources”
- Dropdown: Web
- Dropdown/pill: Fast Research
- Submit arrow button
- Close “X” in top-right of modal

Dropzone area

- Large dashed box: “or drop your files”
- Supported: pdf, images, docs, audio, and more (match wording as closely as you want)

Quick action buttons row

- Upload files
- Websites
- Drive (if you want Google Drive, you’ll need OAuth + Drive API; if you don’t want that complexity, label it “Cloud drive” and implement later)
- Copied text (pasted text)

Each action opens a subflow:

1) Upload files

- file picker
- upload progress
- create source records

2) Websites

- URL input
- fetch + extract readable text
- create web source

3) Drive

- OAuth connect (optional)
- file picker from drive
- import

4) Copied text

- textarea paste
- title field optional
- create pasted-text source

Exact feature alignment with your PDF “governing rules”

To remain compliant with the PDF you gave:

- No premium references anywhere in UI text
- All Studio tools always visible and usable
- Strictly grounded answers and outputs
- Citations always included
- If not in sources: assistant must say info is not available

What I need next to make the replica even closer

Upload 2 more images (if you can) and I'll extend the spec to those screens:

- 1) A chat with sources already loaded (show citations UI)
- 2) A Studio output opened (e.g., mind map or flashcards view)

With those, I can specify:

- the exact citation chip placement style
- how Studio outputs are displayed (cards vs document view)
- thread switcher placement that looks native to your reference UI

There will be pictures. I will reference each picture. The 1st picture is the answer to #1,

which shows the chat panel. When sources are uploaded, the chat panel describes a summary of the sources. You can then ask questions about anything about the sources. Below the summary, you will also find suggested questions about the sources. The AI uses the sources and only the sources to answer questions. There is also a little icon/emoji that goes with the notebook, you can also edit this with any icon the user prefers. Below the icon, there is the # of sources followed by the title of the notebook. (NOTE: There is NO LIMIT to the amount of sources uploaded). Pictures 2-5 are the audio overview settings. There are more pictures after this

Incorporated: Picture #1 (Chat panel behavior) — replica spec update

Chat panel header (top area)

- Notebook icon/emoji displayed at top-left of the notebook summary area
 - Click icon opens an “Edit icon” picker
 - Supports: emoji + optional custom image URL (if you want parity; emoji-only is simplest and matches your note)
- Under icon:
 - Line 1: “X sources” (dynamic count)
 - Line 2: Notebook title (large text)
- No limit on number of sources
 - Implementation note: the UI must handle large source lists (virtualized list + search)

Auto-summary banner (when sources exist)

When at least 1 source is Ready:

- The chat panel shows an AI-generated notebook summary describing the sources (like your screenshot)
- This is not a “chat reply” to a user message; it’s a notebook-level generated overview card

Data model addition (recommended)

- notebook_overviews
 - notebook_id
 - content (markdown)
 - citations (chunk_ids)
 - created_at / updated_at
- Regeneration triggers:
 - Auto-generate after first ingestion completes
 - Re-generate when new sources are added (either automatically or via “Refresh overview” button)

Suggested questions (below the summary)

Below the notebook summary, show 3–6 clickable suggested questions:

- Each suggestion sends a new user message in the active thread
- Suggestions are derived strictly from the sources (RAG)
- Suggestions regenerate whenever the notebook overview regenerates

Suggested question UI (matching picture)

- Stack of pill/rounded rectangles
- Hover highlight
- Clicking inserts and submits immediately

Composer (bottom input bar)

- Placeholder: “Start typing...”
- Right-side:
 - “X sources” indicator (mirrors screenshot)
 - Send arrow button
- Enabled only if there is at least 1 Ready source (per your requirement)

“Save to note” and feedback actions

On the summary card, include the action row:

- “Save to note” button
 - Creates a Studio “Note” output populated with the summary content + citations
- Feedback icons:
 - Copy button (copies summary markdown)
 - Thumbs up / thumbs down (stores feedback event linked to overview_id)

Incorporated: Pictures #2–#5 (Audio Overview settings modal) — replica spec update

Entry point

- Studio pane button: “Audio Overview”

- Clicking opens a modal titled: “Customize Audio Overview”
- Modal has close “X” top-right
- Primary CTA: “Generate” bottom-right (blue pill button)

Modal sections and exact controls

1) Format (card selector)

Four selectable cards in a row:

- Deep Dive
 - “A lively conversation between two hosts, unpacking and connecting topics in your sources”
- Brief
 - “A bite-sized overview to help you grasp the core ideas from your sources quickly”
- Critique
 - “An expert review of your sources, offering constructive feedback to help you improve your material”
- Debate
 - “A thoughtful debate between two hosts, illuminating different perspectives on your sources”

Behavior

- Exactly one selected at a time
- Selected card shows a checkmark in top-right
- Cards are large, rounded, slightly raised on hover

2) Choose language (dropdown)

- Default: English
- You can support more languages later; but the UI should match the dropdown presence now

3) Length (segmented control)

- Short | Default | Long
- Exactly one selected at a time
- Default selected initially

4) Focus prompt (large textbox)

Label: “What should the AI hosts focus on in this episode?”

- Large multiline input with placeholder text
- Includes a small “Tab →” hint shown near the top-right of the text area (as in screenshot)
- Includes a small “magic”/generate icon button on the right side of the box (green icon in your screenshots)
 - Clicking this auto-suggests a focus prompt based on sources (strict RAG)

5) Focus chips (suggested prompts)

Below the textbox, show 3–4 clickable chips (examples in your screenshots like “+ Upgrade Roadmap”, “+ Beginner Overview”)

- Clicking a chip appends that phrase into the textbox (does not overwrite)

Output behavior (per your PDF rules)

- “Audio Overview” output is text-only, as a dialogue between two hosts
- Must be grounded only in sources with citations (even if NotebookLM’s UI doesn’t visibly show them, your PDF requires them)
- Save the generated output in Studio outputs, with:
 - format, language, length, focus_prompt, included_sources

Two clarifications (so the replica stays consistent)

1) Should the notebook summary card appear in every thread, or only once as a notebook-level “pinned” item above threads?

- Best match to your picture: notebook-level pinned item at top of chat view, independent of thread.

2) Do you want citations visibly shown in the chat like chips/footnotes, or should citations be hidden by default but available via a “Sources” toggle?

- Your PDF requires citations; we can keep them visible but subtle (chips that expand).

Send the next pictures when ready (you said there are more). If you label them “Picture #6, #7...” and say what each shows, I’ll keep extending the replica spec screen-by-screen.

All pictures in chat:

WP Certification Prep

Create notebook

Share

Settings

Sources

+ Add sources

Search the web for new sources

Web

Fast Research

Select all sources

(WCE) WordPress Certified Editor - Knowledge Pillars

✓

CIW Certification - Practice Exam Questions with Verifi...

✓

CIW Site Development Associate: Tips for Test Success...

✓

Home Education Program Guidelines and Procedures - ...

✓

Knowledge Pillars - Certipoint - Pearson VUE

✓

Last-Minute Study Tips - Blogs@UWW - UW-Whitewater

✓

Recommended Learning Resources | Knowledge Pillars

✓

Web Certified Editor - for the WordPress® Platform - K...

✓

WordPress Certified Editor Exam Link - Knowledge Pilla...

✓

WordPress Key Concepts Q&A: Comprehensive Guide f...

✓

Chat

post?"), and the other should be the definition or step-by-step action based on Units 1 through 11) 3 - - ."

4. Quiz Prompt

"Generate a quiz with exactly 35 questions to mimic the actual WCE exam format 1 2 . The quiz should be **medium-to-hard** difficulty and cover all 11 units proportionately 3 - - . Include multiple-choice and matching questions that test both conceptual knowledge (like **Copyright and Licensing**) and practical application (like identifying the purpose of a **Spacer block** or how to fix a 'featured image' display error) 17 - - ."

5. Study Guide (Tailored Report) Prompt

"Create a comprehensive study guide for a student who has not yet reviewed the material for the **WCE WordPress Certification** 30 21 . Organize the report by the **11 Unit Objective Domains** 3 - - . For each unit, list the top 3 most critical 'Applied Skills' the student must be able to perform in a live environment 1 14 . Include a 'Last-Minute Success' section with tips on **time management** (1 minute per question), **system requirements** (using Chrome), and **maintenance basics** like 2FA and backups 3 - - ."

Would you like me to go ahead and generate any of these artifacts for you right now? I can start with the Study Guide or the Quiz to help you get a jump on your test prep!

Save to note

Start typing...

10 sources

Studio

Audio Overview

Video Overview

Mind Map

Reports

Flashcards

Quiz

Data Table

WCE WordPress Certified Editor: Comprehensive...

Create Your Own · 10 sources · 1m ago

WordPress Quiz

10 sources · 1m ago

WordPress Flashcards

10 sources · 1m ago

Generating Video Overview...

This may take a while

Generating Audio Overview...

Come back in a few minutes

Add note

1. Audio Overview Prompt

"Generate a **deep-dive** audio overview focusing on the transition of WordPress from a blogging tool to a block-based CMS **3** . The hosts should discuss the 'can-do' vs. 'know' philosophy of Knowledge Pillars' exams **1** and walk through the 11 key objective domains, specifically highlighting the differences between **Posts and Pages**, and the importance of **Categories vs. Tags** **3** ... End with a summary of the exam's structure: 35 questions in 45 minutes with a 75% passing score **1** **2** ."

2. Video Overview Prompt

"Create an **explainer** video overview that focuses on the **WordPress interface and the Block Editor** **3** **7** . The host should visually explain how the 'live-in-the-app' assessment works, demonstrating tasks like navigating the **Media Library**, managing **Comments**, and using **Site Health** tools **1** ... Ensure the video covers the 'Design vs. Write' modes and how to use the **List View** for efficient editing **3** **7** ."

3. Flashcards Prompt

"Create a **standard** set of **hard** difficulty flashcards focusing on the **essential vocabulary and technical requirements** found in the objective domains **3** **15** . Include terms like: **GPL licensing**, **Global Styles**, **Query Loops**, **Permalinks**, **ARIA**, and **Core Web Vitals** **3** ... One side should be the term or a specific task (e.g., 'How to restore a previous version of a post?'), and the other should be the definition or step-by-step action based on Units 1 through 11 **3**"

4. Quiz Prompt

"Generate a quiz with exactly **35 questions** to mimic the actual WCE exam format **1** **2** . The quiz should be **medium-to-hard** difficulty and cover all 11 units proportionately **3** Include multiple-choice and matching questions that test both conceptual knowledge (like **Copyright and Licensing**) and practical application (like identifying the purpose of a **Spacer block** or how to fix a 'featured image' display error) **17**"

5. Study Guide (Tailored Report) Prompt

"Create a comprehensive study guide for a student who has not yet reviewed the material for the **WCE WordPress Certification** **20** **21** . Organize the report by the **11 Unit Objective Domains** **3** For each unit, list the top 3 most critical 'Applied Skills' the student must be able to perform in a live environment **1** **14** . Include a 'Last-Minute Success' section with tips on **time management** (1 minute per question), **system requirements** (using Chrome), and **maintenance basics** like 2FA and backups **3**"

Intelligenza Artificiale Gen...



02:32 / 05:51 • italiano

You did it! Quiz complete.



Right 2
Wrong 10

Topics covered

- Definitions of AI, Machine Learning, and Deep Learning
- History of AI development
- Generative AI in Education
- Traditional vs. AI Programming
- Educational AI tools (Khanmigo, Gemini, NotebookLM, Adobe Express)
- Evaluation framework for AI tools
- Ethical considerations and bias in AI

Need more topics?
You can always generate a new quiz.

Keep learning

Select follow-up topics below and generate a new quiz.

AI Ethics and Bias in depth

Advanced Prompt Engineering for Educators

Implementing AI in Project-Based Learning

Data Privacy and AI compliance in schools

Generate quiz

No questions answered.



Right	0
Wrong	0
Skipped	11

Review Quiz

Retake Quiz



What is the fundamental way projects are organised in NotebookLM, and what is a key limitation of this structure?

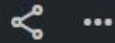
- A. Projects are organised into Notebooks, and information cannot be accessed across different notebooks simultaneously.
- B. Projects are organised in the Studio panel, which has a limit of 1,000 notes per project.
- C. Projects are organised into Sources, which can't be edited once uploaded.
- D. Projects are organised by Chat history, and the history is cleared every 24 hours.

Hint ▾

Next

Birth of the Bill of Rights

Based on 1 source



NotebookLM

Birth of the Bill of Rights



00:00

05:31

1X

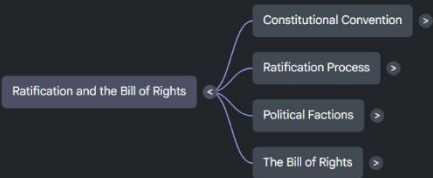


Good video

Bad video

Foundations of the American Constitutional Order

Based on 1 source



Good content

Bad content

Summary of the Bill of Rights Amendments

Based on 1 source

Summary of the Bill of Rights Amendments

Amendment Number	Topic or Right Guaranteed	Key Protections Provided	Historical Context (Inferred)	Source
1	Freedoms of Expression and Religion	Freedom of speech, press, religion, and the right to assemble peacefully, protest, and demand changes	Fear of a strong central government suppressing individual liberties and religious practices as experienced under British rule.	[1]
2	Right to Bear Arms	Not in source	Concern that a federal government might disarm state militias to impose tyranny.	[1]
3	Quartering of Soldiers	Not in source	Grievance against the British practice of forcing colonists to house soldiers in private homes.	[1]
4	Search and Seizure	Protection against unreasonable searches, seizures of property, and arrest	Reaction to British 'writs of assistance' which allowed general, non-specific searches of colonial property.	[1]
5	Due Process	Guarantees due process of law in all criminal cases	Fear of arbitrary imprisonment and unfair legal proceedings by a powerful executive.	[1]
6	Right to a Fair Trial	Guarantees the right to a fair and speedy trial	Desire to prevent the government from holding citizens indefinitely or conducting secret trials.	[1]
7	Trial by Jury in Civil Cases	Not in source	Concern that the new federal courts would ignore local customs and deny citizens a jury of their peers.	[1]
8	Cruel and Unusual Punishment	Protection against cruel and unusual punishment	Fear of the government using torture or excessive penalties to intimidate political opponents.	[1]
9	Non-Enumerated Rights	Not in source	The need to ensure that listing specific rights did not imply that other, unlisted rights were not protected.	[1]
10	Rights Reserved to States or People	Not in source	Antifederalist demand to limit federal power and preserve the sovereignty of the states and the people.	[1]

[1] Ratification of Bill of Rights Readings.docx

Good content

Bad content

Study Guide: The Ratification of the United States Constitution ...

Based on 1 source

Study Guide: The Ratification of the United States Constitution and the Bill of Rights

This study guide provides a comprehensive overview of the transition from the Constitutional Convention to the ratification of the United States Constitution. It explores the ideological divides between the Federalists and Antifederalists, the specific concerns regarding centralized power, and the eventual codification of civil liberties in the Bill of Rights.

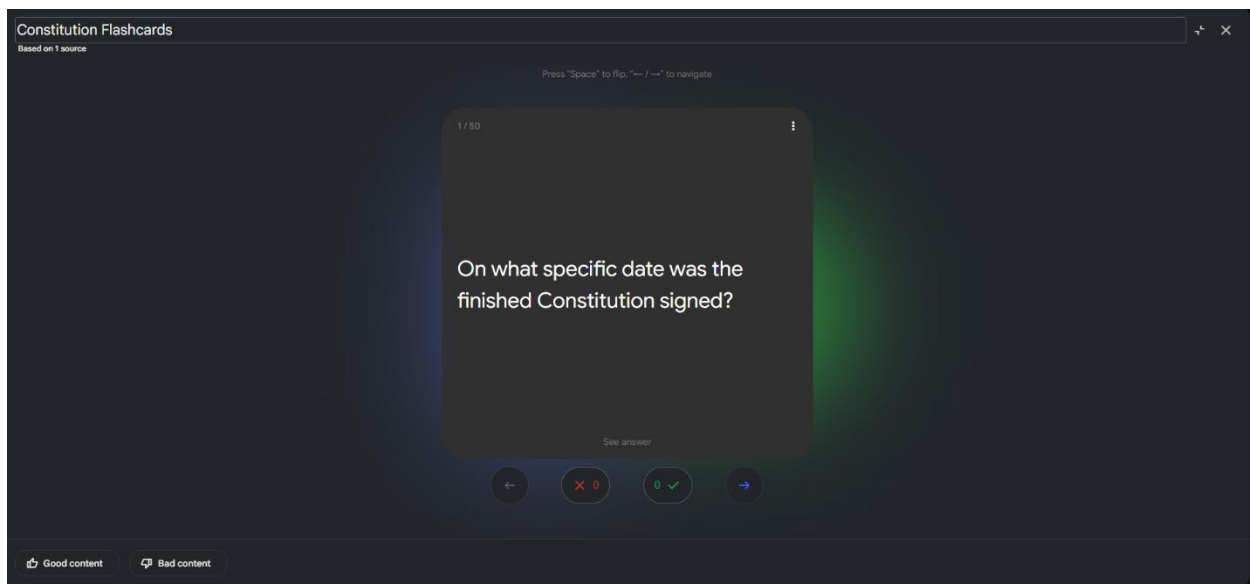
Part I: Short-Answer Quiz

Instructions: Answer the following questions in 2-3 sentences based on the provided source context.

1. What occurred on September 17, 1787, regarding the U.S. Constitution?
2. How did Benjamin Franklin describe the sun on the back of George Washington's chair, and what did this symbolize?
3. What was the legal requirement for the Constitution to become effective after the convention concluded?
4. Despite nine states ratifying by June 1788, why was the support of Virginia and New York considered essential?
5. What specific fears did many citizens have regarding the establishment of a strong central government?
6. How did the Federalists and Antifederalists differ in their vision for the American

Good report

Bad report



What are the key features and requirements of WordPress 7.0?

WordPress 7.0, arriving in 2026, marks the start of **Phase 3 of the Gutenberg project**, which is dedicated to **Collaboration** ¹ ² . While originally scheduled for April 9, 2026, the release was delayed to extend the testing phase and ensure "extreme stability" for its complex new features ¹

Key Features of WordPress 7.0

The primary goal of this release is to transform WordPress into a team-ready publishing platform by integrating workflows that previously required external tools ² .

- **Real-Time Collaboration:** This is the headline feature, allowing multiple users to edit the same post or page simultaneously with live updates ⁵ . It includes visual revision comparisons for side-by-side version reviews and an **AI Client** feature ⁵ .
- **Modernized Admin Dashboard:** The back-end interface, called **DataViews**, which replaces traditional WP List Table, offering a more consistent interface ⁶ . This update includes smooth transitions and a refreshed default color scheme ⁷ .
- **AI Connectors UI:** A new dashboard page under Settings to manage external AI connections in one central location ⁸ . This includes the **AI Client**, a developer framework providing a standard interface for AI integrations across the ecosystem ⁸ .
- **Enhanced Block Editor Tools:** Several new blocks and editing tools to increase design flexibility ⁷ ⁹ :
 - **New Blocks:** Breadcrumbs and Icons blocks ⁹ .

Start typing...

1. Real-Time Collaboration

The headline feature of 7.0 allows multiple users to edit the same post or page simultaneously, with live updates visible across all open sessions. For teams that currently draft content in Google Docs and paste it into the WordPress editor, this offers a native alternative that eliminates that extra step, though how well it fits a given team's workflow will depend on implementation and hosting environment.

What is included in real-time collaboration:

- Multi-user co-editing with live syncing
- Visual revision comparisons for side-by-side version review

Contributor Day – WordCamp Europe 2026



Source guide

This informational webpage outlines the details for **Contributor Day at WordCamp Europe 2026**, a collaborative event scheduled for June 4th in **Kraków, Poland**. The text emphasizes that the gathering is designed for **all skill levels**, inviting developers, designers, and non-technical enthusiasts to give back to the **open-source WordPress project**. The source is structured to guide potential attendees through the **benefits of joining**, such as networking and skill-building, while providing a comprehensive list of diverse **contributor teams** ranging from coding and security to marketing and translation. Ultimately, the document serves as both a **call to action** for community involvement and a practical manual on how to prepare for a day of collective digital stewardship.

Contributor ...

WordCamp Europe 2...

WordPress Open Sou...

Community Collaborat...

Technical Teams Participat...

Contributor Day – WordCamp Europe 2026

[Skip to content](#)

- [Blog](#)
- [About](#)
 - [WordCamp Europe 2026](#)
 - [Contributor Day](#)
 - [Spread the Word](#)
 - [WCEU for Agencies](#)
- [Community](#)
 - [Organisers](#)
 - [Media Partners](#)
 - [Grab Your Badge](#)

Topic	Release Date or Version	Key Features	Technical Requirements	Status or Milestone	Source
WordPress 7.0	April 9, 2026 (Delayed)	Phase 3 Collaboration: Real-time co-editing, modernized admin interface (DataViews), AI Client and Abilities API framework, new native blocks (Breadcrumbs, Icons), and grid block responsiveness.	Minimum PHP 7.4 (PHP 8.3+ recommended), WebSocket support for real-time collaboration features.	Delayed	[1, 2]
Full Site Editing (FSE)	Standard by 2026	Block Bindings API for dynamic data, fluid typography, theme.json v3 schema, synced patterns, and visual design of global styles/templates.	Block-based theme (HTML templates), theme.json configuration, WordPress 6.7+ or 7.0.	Stable	[3]
Advanced Professional WordPress Developer Certification	February 19, 2026	Standardized benchmark for enterprise-grade skills in scalability, security, performance, and disaster recovery; 120-minute multiple-choice exam; 3-year validity.	Platform-agnostic; \$250 USD registration fee; online-proctored or in-person delivery.	Released	[4]
WP-CLI	2026	AI-powered commands (ai-generate-alt, ai-summarize, ai-optimize-code), vulnerability scanning, smart path-mapping for server transfers, and S3 remote storage integration.	PHP environment compatible with Phar archive; shell access (SSH).	Stable	[5]
[1] WordPress 7.0: New Features and What to Expect [2] WordPress Delays Release Of Version 7.0 To Focus On Stability [3] WordPress Full Site Editing (FSE): Complete Guide 2026 [4] Introducing the Advanced Professional WordPress Developer Certification – WordPress VIP Customer Hub [5] WP-CLI in 2026: New Commands and Features for Modern Admins - DoHost					

What are the key features and requirements of WordPress 7.0?

WordPress 7.0, marking the start of Gutenberg Phase 3: Collaboration, was originally scheduled for release on April 9, 2026, but was delayed by several weeks to prioritize "extreme stability," particularly regarding its new real-time features. [↗](#) [↖](#)

Key Features

Real-Time Collaboration: This is the headline feature of Phase 3, allowing multiple users to edit a post or page simultaneously with live syncing. [↗](#) [↖](#) It includes multi-user co-editing, visual revision comparisons for side-by-side reviews, and an improved Notes feature with real-time syncing and a dedicated keyboard shortcut. [↗](#) [↖](#)

Modernized Admin Dashboard: The back-end interface has been refreshed using a system called DataViews, which replaces traditional WP List Tables with a more modern, consistent interface. [↗](#) [↖](#) Improvements include a cleaner visual layout, refreshed color schemes, and smoother cross-document view transitions. [↗](#) [↖](#)

AI Developer Framework (Abilities API): Instead of adding a native AI writing tool, WordPress 7.0 introduces the Abilities API and AI Client. [↗](#) [↖](#) This framework allows developers to build consistent AI-powered features across the ecosystem. [↗](#) [↖](#) A new Connectors UI dashboard under "Settings" enables site owners to manage these external AI connections in one place. [↗](#) [↖](#)

Enhanced Block Features: Several core blocks received significant updates:

- Navigation Block: Now includes customizable overlays with specific mobile breakpoint controls. [↗](#) [↖](#)
- New Blocks: Native Breadcrumbs and Icons blocks have been added, reducing the need for external plugins. [↗](#) [↖](#)
- Grid Block: Now responsive-enabled, allowing layouts to adapt to various screen sizes. [↗](#) [↖](#)
- Cover Block: Supports video embeds as section backgrounds. [↗](#) [↖](#)

Full Site Editing (FSE) Improvements: The update brings more granular design tools for animations and transitions, as well as section-level styling, allowing different Global Styles to be applied to specific zones within a single template. [↗](#) [↖](#)

Requirements and Technical Constraints

PHP Version Requirements: WordPress 7.0 raises the minimum technical requirements significantly.

- Minimum Supported: PHP 7.4. [↗](#) [↖](#)
- Recommended: PHP 8.3 or higher for optimal performance and security. [↗](#) [↖](#)
- Note: Sites running on PHP 7.2 or 7.3 will not receive this update and will remain on the 6.9 branch. [↗](#) [↖](#)

WebSocket Support: The real-time collaboration features may require WebSocket support from your hosting provider. [↗](#) [↖](#) If a host does not support WebSockets, these team workflows may not function. [↗](#) [↖](#)

Performance Concerns: During development, it was noted that active real-time editing sessions currently disable persistent post caches, a performance issue the core team is working to resolve. [↗](#) [↖](#) There are also ongoing discussions about whether to use a single or separate database tables to handle high-frequency writes for real-time syncing. [↗](#) [↖](#)

Preparation Recommendations

Due to the complexity of the new database schema and the admin refresh, site owners are encouraged to test the update in a staging environment first. [↗](#) [↖](#) It is also recommended to audit existing plugins, as those that hook into traditional WP List Tables may require updates to remain compatible with the new DataViews interface. [↗](#) [↖](#)

📌 Save to note

🔗

📄

🔖

💬

How can I check if my host supports WebSockets?

What is the new DataViews system in the admin dashboard?

Tell me more about the Abilities API for AI developers.



Contributor Day – WordCamp Europe 2026



Remove source ✖



Rename source /e

Sources



+ Add sources

🔍 Search the web for new sources



Web



Fast Research



Select all sources



Contributor Day – WordCamp Europe 2026



Fix Critical WordPress Errors Step by Step (2026) | Pro...



Introducing the Advanced Professional WordPress Dev...



Version 6.9.4 – Documentation – WordPress.org



WP Engine Review 2026: Can You Still Use It With Word...



WP-CLI in 2026: New Commands and Features for Mod...



WordPress 7.0: New Features and What to Expect



WordPress Delays Release Of Version 7.0 To Focus On ...



WordPress Full Site Editing (FSE): Complete Guide 2026



<https://centralina.org/wp-content/uploads/2025/11/WC...>





Customize Data Table



Choose language

English



Describe the data table you want to create

Things to try

- Create a table with the major findings in these research papers, using columns: title, author, key result
- Extract the most important quotes from my readings, grouping them by topic and author
- List vacation destinations in Italy with city, best time to visit, attractions, and cost

Generate



Customize Quiz



Number of Questions

Fewer

✓ Standard (Default)

More

Level of Difficulty

Easy

✓ Medium (Default)

Hard

What should the topic be?

Things to try

- The quiz must be restricted to a specific source (e.g. "the article about Italy")
- The quiz must focus solely on the key concepts of physics
- Create a quiz to help me prepare for my history exam on Ancient Egypt



Generate

Customize Flashcards



Number of Cards

Fewer

✓ Standard (Default)

More

Level of Difficulty

Easy

✓ Medium (Default)

Hard

What should the topic be?

Things to try

- The flashcards must be restricted to a specific source (e.g. "the article about Italy")
- The flashcards must focus on a specific topic like "Newton's second law"
- The card fronts must be short (1-5 words) for memorization



Generate

Create report



Format

Create Your Own

Craft reports your way by specifying structure, style, tone, and more

Briefing Doc



Overview of your sources featuring key insights and quotes

Study Guide



Short-answer quiz, suggested essay questions, and glossary of key terms

Blog Post



Insightful takeaways distilled into a highly readable article

✦ Loading suggestions...





Customize Video Overview



Choose language

English (default)



What should the AI host focus on?

Things to try

- Target a specific use case ("present this to a book club", "help me review for a quiz")
- Focus on a specific source ("show the photos from the album", "focus on the biology paper")
- Describe the show structure ("start by talking about the mission", "end with next steps")



Generate

Customize Audio Overview



Format

Deep Dive

A lively conversation between two hosts, unpacking and connecting topics in your sources

Brief

A bite-sized overview to help you grasp the core ideas from your sources quickly

Critique

An expert review of your sources, offering constructive feedback to help you improve your material

Debate



A thoughtful debate between two hosts, illuminating different perspectives on your sources

Choose language

English



Length

Short



Default

What should the AI hosts focus on in this episode?

*Debate the value of the new WordPress Developer Certification for the broader industry.
- Focus on objective validation versus subjective hiring.*



+ Certification Debate

+ FSE vs Code

+ Expert Panel

Generate

Customize Audio Overview



Format

Deep Dive



A lively conversation between two hosts, unpacking and connecting topics in your sources

Brief

A bite-sized overview to help you grasp the core ideas from your sources quickly

Critique



An expert review of your sources, offering constructive feedback to help you improve your material

Debate

A thoughtful debate between two hosts, illuminating different perspectives on your sources

Choose language

English



Length

Short



Default

What should the AI hosts focus on in this episode?

Evaluate the material for an audience of junior WordPress developers seeking certification or advanced FSE skills.



+ Technical Skill Check

+ Structural Flow Analysis

+ Engagement and Tone

Generate

Format

Deep Dive

A lively conversation between two hosts, unpacking and connecting topics in your sources

Brief



A bite-sized overview to help you grasp the core ideas from your sources quickly

Critique

An expert review of your sources, offering constructive feedback to help you improve your material

Debate

A thoughtful debate between two hosts, illuminating different perspectives on your sources

Choose language

English

What should the AI hosts focus on in this episode?

Create a simple step-by-step preparation list for the WordPress 7.0 update.



+ Upgrade Checklist

+ Developer Focus

+ Beginner Overview

Generate

Format

Deep Dive



A lively conversation between two hosts, unpacking and connecting topics in your sources

Brief

A bite-sized overview to help you grasp the core ideas from your sources quickly

Critique

An expert review of your sources, offering constructive feedback to help you improve your material

Debate

A thoughtful debate between two hosts, illuminating different perspectives on your sources

Choose language

English



Length

Short



Default

Long

What should the AI hosts focus on in this episode?

Outline a step-by-step checklist for upgrading mission-critical sites to WordPress 7.0.

Tab →



+ Upgrade Roadmap

+ Developer Certification

+ Modern Theme Building

Generate



Geo EOC

7 sources

thought These educational materials explore fundamental **geometric principles** and the various ways shapes can be manipulated on a **coordinate plane**. The sources define core building blocks like **points, lines, and planes**, while providing formulas for calculating **segment lengths**, midpoints, and weighted averages. Detailed tutorials explain **transformations**, including how to **dilate** objects from centers other than the origin by using scale factors to adjust distances. Additionally, the texts cover **reflections** across mirror lines and **rotations** around central points, noting how these moves affect an image's orientation. Finally, the concept of **symmetry** is introduced, describing how figures can remain identical after being flipped, turned, or slid.



Save to note



How do I dilate shapes when the center isn't the origin?

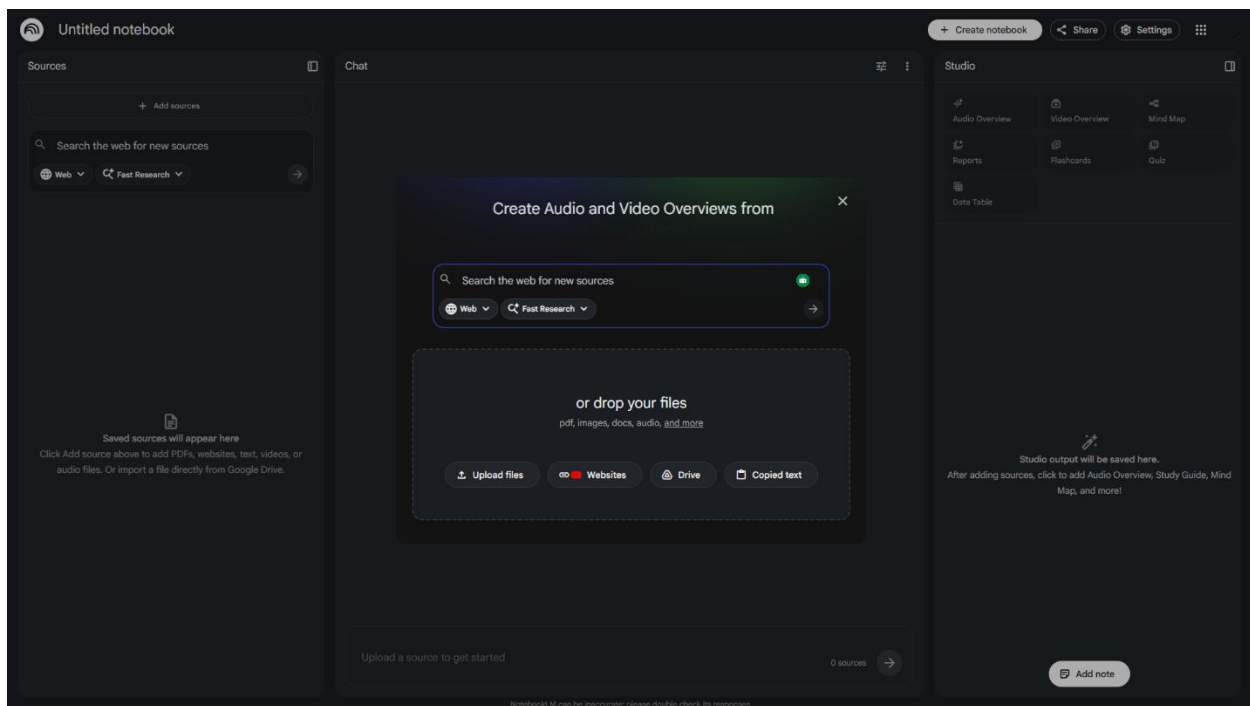
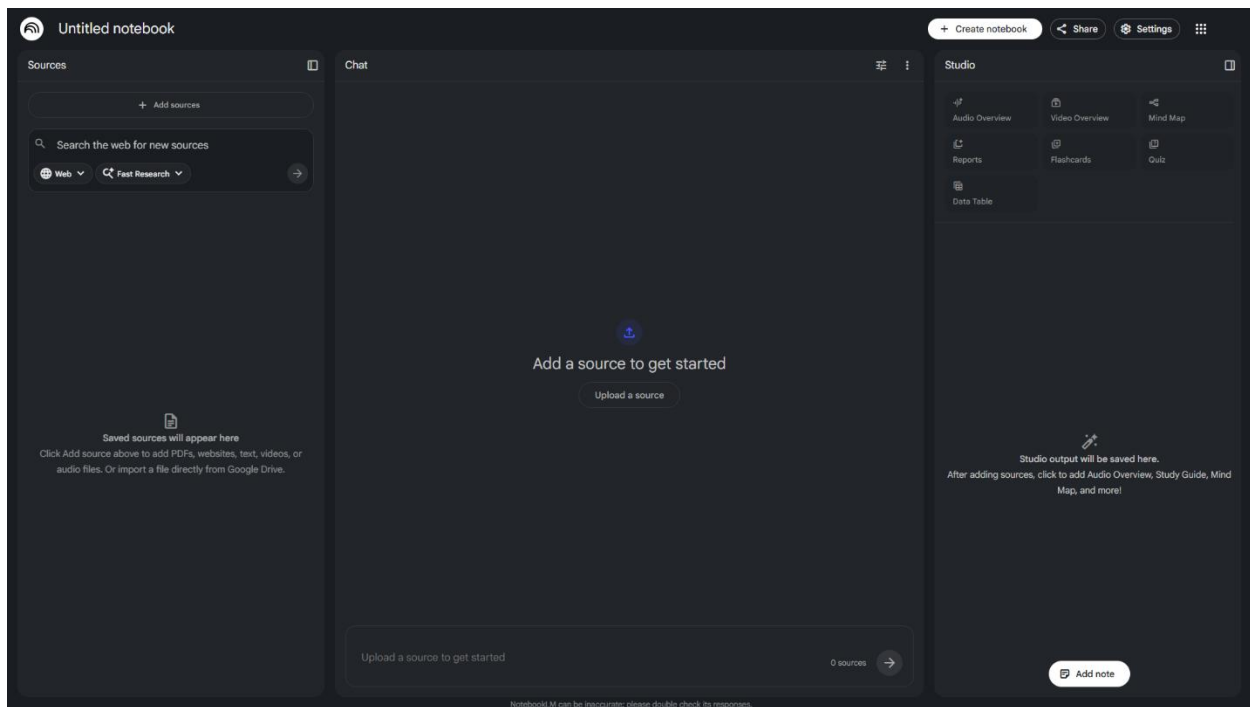
What is the difference between reflection and rotational symmetry?

How do weighted averages help locate points on a line?

Start typing...

7 sources





All My notebooks Featured notebooks



Most recent

+ Create new

My notebooks



Create new notebook